Tarea 2 - ECP ESP

Entrega única para los dos módulos. Los estudiantes deben subir **el mismo archivo** `.py` a cada módulo de Moodle.

Instrucciones

1. Descarga la plantilla `tarea2_APELLIDO_NOMBRE.py` y renómbrala con tus datos.
2. Completa los 10 ejercicios en ese archivo (cada enunciado está en un bloque de comentarios).
3. Asegúrate de que el script se ejecute sin errores de principio a fin.
4. Sube el archivo `.py` a los dos módulos de Moodle (ECP y ESP).

Ejercicios

Ejercicio 1 – Gestión de lista de compras

- Crea una lista vacía llamada `lista_compras`.
- Usa un bucle `while` para pedir productos al usuario hasta que escriba "fin".
- Muestra la lista final y el número total de productos.
- Usa los métodos `append()` y `len()`.

```
Ejemplo:
Producto (o 'fin' para terminar): leche
Producto (o 'fin' para terminar): pan
Producto (o 'fin' para terminar): huevos
Producto (o 'fin' para terminar): fin

Lista de compras: ['leche', 'pan', 'huevos']
Total de productos: 3
```

Ejercicio 2 – Análisis de temperaturas semanales

- Dada la lista `temperaturas = [22.5, 19.0, 25.3, 28.1, 24.0, 18.5, 21.0]` (temperaturas de lunes a domingo):

- Calcula e imprime: temperatura máxima, mínima y promedio (con 2 decimales).
- Cuenta cuántos días superaron los 23 grados (días calurosos).
- Muestra qué días de la semana fueron calurosos (usa una lista de nombres de días).
- Usa las funciones `max()`, `min()`, `sum()`, `len()` y un bucle `for` con `enumerate()`.

```
Ejemplo de salida:
Temperatura máxima: 28.1 °C
Temperatura mínima: 18.5 °C
Promedio semanal: 22.63 °C
Días calurosos (>23°C): 3
Días calurosos: Miércoles, Jueves, Viernes
```

Ejercicio 3 – Generador de serie numérica

- Pide al usuario un número inicial, un número final y un incremento.
- Usa un bucle `for` con `range()` para generar la serie.
- Calcula y muestra la suma de todos los números de la serie.
- Valida que el número inicial sea menor que el final.

```
Ejemplo:
Número inicial: 5
Número final: 25
Incremento: 5

Serie generada: 5, 10, 15, 20
Cantidad de números: 4
Suma total: 50
```

Ejercicio 4 – Búsqueda en lista con while

- Dada la lista `ciudades = ["Madrid", "Barcelona", "Valencia", "Sevilla", "Bilbao"]`.
- Pide una ciudad al usuario y busca si está en la lista usando un bucle `while`.
- Si la encuentra, muestra su posición (índice) y usa `break` para salir.
- Si no la encuentra, muestra un mensaje apropiado (usa `else` del bucle).

```
Ejemplo 1:
Ciudad a buscar: Valencia
Valencia encontrada en la posición 2

Ejemplo 2:
Ciudad a buscar: Málaga
Málaga no está en la lista
```

Ejercicio 5 – Diccionario de películas favoritas

- Crea un diccionario `peliculas` donde cada clave es el título y el valor es otro diccionario con `año`, `género` y `valoración` (del 1 al 10).
- Añade al menos 4 películas iniciales.
- Implementa las siguientes funciones:
  - `agregar_pelicula(titulo, año, genero, valoracion)` : añade una película.
  - `peliculas_por_genero(genero)` : devuelve lista de títulos de ese género.
  - `mejor_valorada()` : devuelve el título de la película con mayor valoración.
- Muestra el catálogo completo con un bucle `for` .

```
Ejemplo:
--- Catálogo de películas ---
Inception (2010) - Ciencia ficción - 9/10
El Padrino (1972) - Drama - 10/10
Toy Story (1995) - Animación - 8/10
Matrix (1999) - Ciencia ficción - 9/10

Películas de Ciencia ficción: ['Inception', 'Matrix']
Mejor valorada: El Padrino (10/10)
```

Ejercicio 6 – Análisis de ventas por categoría

- Dada la siguiente lista de ventas (cada venta es una tupla con categoría y monto):

```
ventas = [
    ("Electrónica", 450),
    ("Ropa", 120),
    ("Electrónica", 280),
    ("Hogar", 95),
    ("Ropa", 200),
    ("Electrónica", 175),
    ("Hogar", 310),
    ("Ropa", 85)
]
```

- Usa un diccionario para acumular el total de ventas por categoría.
- Muestra el resumen de ventas por categoría.
- Indica cuál es la categoría con mayores ventas totales.
- Calcula el promedio de venta por transacción.

```
Ejemplo de salida:
Resumen de ventas por categoría:
Electrónica: 905 €
Ropa: 405 €
Hogar: 405 €

Categoría líder: Electrónica (905 €)
Total de transacciones: 8
Promedio por transacción: 213.75 €
```

Ejercicio 7 – Comparador de listas de reproducción

- Define dos conjuntos que representen canciones de dos playlists:

```
playlist_rock = {"Bohemian Rhapsody", "Stairway to Heaven", "Hotel California", "Back in
playlist_favoritos = {"Bohemian Rhapsody", "Billie Jean", "Hotel California", "Thriller"
```

- Calcula y muestra usando operadores de conjuntos (`&` , `-` , `|` , `^`):
  - Canciones que están en ambas playlists.
  - Canciones exclusivas de Rock.
  - Canciones exclusivas de Favoritos.
  - Canciones que están en una u otra, pero no en ambas (diferencia simétrica).
  - Total de canciones únicas en tu biblioteca.

```
Ejemplo:
Canciones en común: {'Bohemian Rhapsody', 'Hotel California'}
Solo en Rock: {'Stairway to Heaven', 'Back in Black', 'Sweet Child O Mine'}
Solo en Favoritos: {'Billie Jean', 'Thriller', 'Imagine'}
En una pero no en ambas: {'Stairway to Heaven', 'Billie Jean', ...}
Total en biblioteca: 8 canciones únicas
```

Ejercicio 8 – Gestión de biblioteca con diccionarios anidados

- Crea el siguiente diccionario de biblioteca:

```
biblioteca = {
    "978-0-13-468599-1": {"titulo": "Python Crash Course", "autor": "Eric Matthes", "año
    "978-1-59327-584-6": {"titulo": "Automate the Boring Stuff", "autor": "Al Sweigart",
    "978-0-596-51774-8": {"titulo": "Learning Python", "autor": "Mark Lutz", "año": 201
    "978-1-491-95036-8": {"titulo": "Fluent Python", "autor": "Luciano Ramalho", "año":
}
```

- Implementa las funciones:
  - `libros_disponibles()` : devuelve lista de títulos disponibles para préstamo.
  - `buscar_por_autor(autor)` : devuelve lista de libros de ese autor.
  - `libro_mas_reciente()` : devuelve tupla (ISBN, título, año) del libro más nuevo.
  - `prestar_libro(isbn)` : cambia `disponible` a False si está disponible.
- Muestra los resultados de cada función.

```
Ejemplo:
Libros disponibles: ['Python Crash Course', 'Learning Python', 'Fluent Python']
Libros de Mark Lutz: ['Learning Python']
Libro más reciente: ('978-1-491-95036-8', 'Fluent Python', 2022)

Prestando libro 978-0-13-468599-1...
Préstamo realizado: Python Crash Course
```

Ejercicio 9 – Procesamiento de tuplas

- Dada la lista de tuplas con datos de estudiantes:

```python
estudiantes = [
    ("Ana", 20, 8.5),
    ("Carlos", 22, 6.0),
    ("María", 21, 9.2),
    ("Juan", 23, 4.5),
    ("Pedro", 20, 7.8)
]
```

- Usa desempaquetado de tuplas en un bucle `for` para:
  - Mostrar nombre y nota de cada estudiante.
  - Calcular el promedio de notas.
  - Crear una lista con los nombres de los aprobados (nota >= 5).
  - Encontrar al estudiante con la nota más alta.

```
Ejemplo:
--- Listado de estudiantes ---
Ana: 8.5
Carlos: 6.0
María: 9.2
Juan: 4.5
Pedro: 7.8

Promedio de notas: 7.20
Estudiantes aprobados: ['Ana', 'Carlos', 'María', 'Pedro']
Mejor estudiante: María con 9.2
```

Ejercicio 10 – Menú interactivo con todas las estructuras

- Crea un programa con un menú que permita gestionar una lista de tareas:
  i. Agregar tarea
  ii. Eliminar tarea
  iii. Marcar tarea como completada
  iv. Mostrar tareas pendientes
  v. Mostrar tareas completadas
  vi. Salir
- Usa un diccionario con dos claves: `"pendientes"` (lista) y `"completadas"` (lista).
- El menú debe repetirse con un bucle `while` hasta que el usuario elija salir.
- Valida las opciones del menú y muestra mensajes apropiados.

```
Ejemplo:
=== GESTOR DE TAREAS ===
1. Agregar tarea
2. Eliminar tarea
3. Marcar como completada
4. Mostrar pendientes
5. Mostrar completadas
6. Salir

Opción: 1
Nueva tarea: Estudiar Python
Tarea agregada correctamente

Opción: 1
Nueva tarea: Hacer ejercicios
Tarea agregada correctamente

Opción: 4
Tareas pendientes:
1. Estudiar Python
2. Hacer ejercicios

Opción: 3
Número de tarea a completar: 1
Tarea 'Estudiar Python' marcada como completada

Opción: 5
Tareas completadas:
1. Estudiar Python

Opción: 6
¡Hasta luego!
```

Rúbrica de evaluación – ESP (Entornos y Sintaxis en Python)

RA5 – Sintaxis, operadores y tipos complejos (100 %)

Nivel	Descripción
2 puntos	Demuestra dominio de la sintaxis para tipos complejos: crea y manipula listas, diccionarios, tuplas y sets correctamente; usa sus métodos principales (`append` , `pop` , `keys` , `values` , `items` , `add` , `remove` , operadores de conjuntos `|` , `&` , `-`); accede a elementos por índice y clave; formatea salidas con f-strings; distingue entre tipos mutables e inmutables.
1 punto	Usa las estructuras de datos con errores puntuales (confusión de métodos, acceso incorrecto, errores en operaciones de conjuntos) o no demuestra comprensión de mutabilidad/inmutabilidad.
0 puntos	No utiliza correctamente las estructuras de datos complejas o el código no ejecuta debido a fallos en su manipulación.

Rúbrica de evaluación – ECP (Estructuras de Control en Python)

RA3 – Sentencias iterativas (50 %)

Nivel	Descripción
2 puntos	Identifica cuándo usar `for` vs `while` ; implementa correctamente ambos tipos de bucle; usa `range()` con sus variantes (inicio, fin, paso); aplica `break` , `continue` y `else` en bucles cuando es apropiado; implementa bucles anidados correctamente; itera sobre listas, diccionarios, tuplas y sets con la sintaxis adecuada.

Nivel	Descripción
1 punto	Usa los bucles con errores menores (condiciones incorrectas, falta de actualización de variable en `while`, uso inadecuado de `range()`) o no aprovecha las características avanzadas (`break`, `continue`, `else`).
0 puntos	No implementa bucles correctamente, produce bucles infinitos, o no sabe elegir entre `for` y `while` según el problema.

RA4 – Funciones definidas por el usuario (50 %)

Nivel	Descripción
2 puntos	Define funciones con `def` para encapsular lógica reutilizable; usa parámetros y `return` correctamente; las funciones tienen una única responsabilidad; valida entradas cuando es necesario; documenta las funciones con docstrings básicos; reutiliza funciones auxiliares para evitar duplicación de código.
1 punto	Define funciones pero mezcla responsabilidades (print dentro de funciones que deberían devolver valores), faltan parámetros necesarios, o no usa `return` adecuadamente.
0 puntos	No define funciones personalizadas o están mal implementadas (no devuelven datos, no se invocan, errores de sintaxis).

Correspondencia ejercicios - RAs

Ejercicio	Contenido principal	ESP RA5	ECP RA3	ECP RA4
1	Lista + while	X	X	
2	Lista + for + enumerate	X	X	
3	for + range() con parámetros	X	X	
4	while + break/else + lista	X	X	
5	Diccionarios anidados + funciones	X	X	X

Ejercicio	Contenido principal	ESP RA5	ECP RA3	ECP RA4
6	Tuplas + diccionario + acumulador	X	X	
7	Sets + operadores de conjuntos	X		
8	Diccionarios anidados + funciones	X	X	X
9	Tuplas + desempaquetado + for	X	X	
10	Integrador: dict + listas + while + funciones	X	X	X